

Test Case ID: TC-FUNC-MP-001

Objective: Verify successful property making with valid data and image uploads.

Preconditions: The PostgreSQL database and backend server are running. A user account exists, and the user is authenticated (a valid JWT is stored in `localStorage`). **Test**

Procedure:

1. Navigate to the frontend "Make Listing" page (e.g., `/make-listing`).
2. Enter "Modern Downtown Loft" in the Title field.
3. Enter "Beautiful open-concept loft with city views." in the Description field.
4. Enter "250000" in the Price field.
5. Enter "123 Main St, Istanbul" in the Address field.
6. Attach 2 valid image files (e.g., `loft1.jpg`, `loft2.jpg`) to the file upload input handled by Multer.
7. Click the "Submit" or "Publish" button. **Expected Result:** The frontend makes a `POST` request to `/api/property` with the form data and files. The server successfully parses the request, uploads the images, inserts records into the `properties` and `property_images` tables, and returns a `201 Created` status with the new property data. The frontend displays a success notification and redirects the user to the `/my-listings` dashboard or the newly made property detail page.

Testing Type: Functional

Test Case ID: TC-FUNC-MP-002

Objective: Verify form validation and server response when mandatory property fields are omitted. **Preconditions:** The database and server are running. The user is authenticated.

Test Procedure:

1. Navigate to the frontend "Make Listing" page.
2. Leave the "Title" and "Price" fields empty.
3. Enter valid text into the "Description" and "Address" fields.
4. Click the "Submit" button. **Expected Result:** The frontend UI prevents the submission and highlights the missing required fields (e.g., via HTML5 validation or React state errors). If the frontend validation is bypassed, the Playwright API context directly sending a `POST /api/property` with missing fields should result in the database query failing or the server returning a `400 Bad Request` or `500 Internal Server Error`, and no property is saved to the database.

Testing Type: Functional

Test Case ID: TC-FUNC-MP-003

Objective: Verify that an unauthenticated user cannot access the making form or submit a new property. **Preconditions:** The database and server are running. The browser session has NO authenticated user (no JWT in `localStorage`). **Test Procedure:**

1. Attempt to navigate directly to the "Make Listing" route via the browser URL.
2. Using Playwright's API request context, send a `POST` request to `/api/property` containing valid form data and mock images, but omitting the `Authorization: Bearer <token>` header. **Expected Result:** For step 1, the React frontend router blocks access to the protected route and redirects the user to the Login page. For step 2, the backend authorization middleware intercepts the request and returns a `401 Unauthorized` status before the `makeProperty` controller is ever reached.

Testing Type: Functional

Test Case ID: TC-FUNC-MP-004

Objective: Verify AI description generation successfully populates the description field based on uploaded images. **Preconditions:** The database and server are running. The user is authenticated. A valid Gemini API key is configured on the backend. **Test Procedure:**

1. Navigate to the frontend "Make Listing" page.
2. Attach valid image files to the file upload input.
3. Click the "Generate AI Description" button (which triggers the `POST` to the AI endpoint).
4. Wait for the backend response. **Expected Result:** The frontend sends the files to the AI endpoint. The `generatePropertyDescription` service successfully processes the images using the `gemini-2.5-flash` model and returns a 100-150 word text description. The frontend automatically populates the "Description" textarea with the returned AI-generated text. **Testing Type:** Functional

Test Case ID: TC-FUNC-VIEW-001

Objective: Verify that any user (unauthenticated) can successfully retrieve and view the list of all published properties. **Preconditions:** The PostgreSQL database is running and contains at least two seeded property listings. The user is completely logged out (no JWT).

Test Procedure: 1. Navigate directly to the public frontend URL displaying all properties (e.g., / or /properties). 2. Alternatively, fire an unauthenticated GET request directly to the all-properties API endpoint. **Expected Result:** The server processes the request without triggering authorization middleware and returns a 200 OK status with an array of property data. The frontend successfully maps over the array and renders the property cards. **Testing Type:** Functional

Test Case ID: TC-FUNC-VIEW-002

Objective: Verify that a user can successfully view the details of a specific, existing property using a valid ID. **Preconditions:** A property with a known ID (e.g., 1) exists in the database.

Test Procedure: 1. Navigate to the frontend property detail page (e.g., /property/1).

Expected Result: The server successfully queries the database via GET /api/property/:id, bundles the images using the scalar subquery or COALESCE logic, and returns a 200 OK status. The frontend renders the full details (title, description, price, and image gallery). **Testing Type:** Functional

Test Case ID: TC-FUNC-VIEW-003

Objective: Verify the controller's robust error handling when provided with an invalid ID format (preventing legacy slice logic crashes). **Preconditions:** The backend server is running. **Test Procedure:** 1. Manually navigate to a property detail route using a non-numeric string (e.g., /property/abc).

Expected Result: The propertyController parses the ID and fails the isNaN() check. It safely returns a 400 Bad Request with the message "Invalid Property ID provided" without crashing the Express server. The legacy id.slice(1) logic remains absent and does not interfere. **Testing Type:** Functional

Test Case ID: TC-FUNC-VIEW-004

Objective: Verify the server response when a validly formatted ID is provided, but the property does not exist in the database. **Preconditions:** The database does *not* contain a property with ID 99999. **Test Procedure:** 1. Navigate to `/property/99999`. **Expected Result:** The controller passes the `isNaN()` check but the `propertyService` returns `null`. The server returns a `404 Not Found` status with the message "Property not found". The frontend gracefully displays an empty state or error UI. **Testing Type:** Functional

Objective: Verify successful property update with valid data by the rightful owner.

Preconditions: The database is running. A user exists and is authenticated. A property owned by this user exists in the database. **Test Procedure:** 1. Navigate to the "My Listings" dashboard and click "Edit" on the seeded property. 2. Clear the existing "Title" and "Price" fields. 3. Enter "Updated Luxury Villa" as the new Title and "750000" as the new Price. 4. Click the "Save Changes" button. **Expected Result:** The frontend makes a **PUT** or **PATCH** (or **POST**, depending on your exact routing) request to `/api/property/:id`. The server returns a **200 OK** status with the updated property data. The frontend displays a success message, and the UI correctly reflects the new title and price. **Testing Type:** Functional

Test Case ID: TC-FUNC-EDIT-002

Objective: Verify unauthorized users cannot update properties they do not own.

Preconditions: Two distinct users exist (User A and User B). User A owns a property. User B is authenticated and active. **Test Procedure:** 1. User B logs in. 2. Using Playwright's API request context, User B sends an update request to the API endpoint targeting User A's property ID.

Expected Result: The backend `updateProperty` controller queries the database, sees that `existingProperty.user_id` does not match User B's ID, and immediately returns a **403 Forbidden** status with the message "Unauthorized: You can only edit your own properties." The database remains unchanged. **Testing Type:** Security / Functional

Test Case ID: TC-FUNC-EDIT-003

Objective: Verify server response when attempting to update a validly formatted but non-existent property ID. **Preconditions:** The user is authenticated. No property exists with ID **99999**. **Test Procedure:** 1. Using Playwright's API request context, send an update request to `/api/property/99999` with a valid JWT. **Expected Result:** The controller passes the valid ID to the service. The service returns `null` for the existing property check. The server returns a **404 Not Found** status with the message "Property not found". **Testing Type:** Functional

Test Case ID: TC-FUNC-EDIT-004

Objective: Verify controller's robust error handling when provided with an invalid ID format.

Preconditions: The backend server is running, and the user is authenticated. **Test**

Procedure: 1. Send an update API request to an invalid route string, such as
`/api/property/abc`.

Expected Result: The controller parses the ID, fails the `isNaN()` check, and safely returns a `400 Bad Request` with the message "Invalid Property ID provided". **Testing Type:**
Functional

Test Case ID: TC-FUNC-DEL-001

Objective: Verify successful property deletion by the rightful owner. **Preconditions:** The database is running. A user is authenticated. A property owned by this user exists in the database. **Test Procedure:** 1. Navigate to the "My Listings" dashboard. 2. Locate the seeded property and click the "Delete" button. 3. (If applicable) Confirm the deletion in the UI prompt. **Expected Result:** The frontend makes a `DELETE` request to `/api/property/:id`. The server verifies ownership and executes the deletion, returning a `200 OK` status with a success message. The frontend optimistically removes the item from the dashboard UI. In the database, the property is removed, and all associated rows in `property_images`, `favorites`, and `offers` are cascaded away. **Testing Type:** Functional

Test Case ID: TC-FUNC-DEL-002

Objective: Verify unauthorized users cannot delete properties they do not own. **Preconditions:** Two distinct users exist (User A and User B). User A owns a property. User B is authenticated and active. **Test Procedure:** 1. User B logs in. 2. Using Playwright's API request context, User B sends a `DELETE` request to `/api/property/:id` targeting User A's property ID. **Expected Result:** The backend `deleteProperty` controller queries the database, detects that `existingProperty.user_id` does not match User B's JWT payload ID, and immediately returns a `403 Forbidden` status with the message "Unauthorized: You can only delete your own properties." The property remains untouched in the database. **Testing Type:** Security / Functional

Test Case ID: TC-FUNC-DEL-003

Objective: Verify server response when attempting to delete a non-existent property ID. **Preconditions:** The user is authenticated. No property exists with ID `99999`. **Test Procedure:** 1. Using Playwright's API request context, send a `DELETE` request to `/api/property/99999` with a valid JWT. **Expected Result:** The controller parses the ID and calls the service. The service returns `null` for the existing property check. The server returns a `404 Not Found` status with the message "Property not found". **Testing Type:** Functional

Test Case ID: TC-FUNC-DEL-004

Objective: Verify the controller's robust error handling when provided with an invalid ID format. **Preconditions:** The backend server is running, and the user is authenticated. **Test Procedure:** 1. Send a **DELETE** API request to an invalid route string, such as `/api/property/abc`. **Expected Result:** The controller parses the ID, fails the `isNaN()` check, and returns a **400 Bad Request** with the message "Invalid property ID format" before ever hitting the database. **Testing Type:** Functional

Test Case ID: TC-FUNC-FAV-001

Objective: Verify an authenticated user can successfully save a property to their favorites.

Preconditions: The database is running. A user is authenticated. A valid property exists in the database. The property is not currently saved by the user. **Test Procedure:** 1. Navigate to the frontend property detail page or search list. 2. Click the "Save" (Heart) button on the specific property.

Expected Result: The frontend makes a **POST** request to `/api/property/:id/favorite` with the Bearer token. The backend `saveProperty` controller executes and inserts a record into the `favorites` junction table using the user's ID and property ID. The server returns a **200 OK** status with a success message. The UI updates optimistically to show the "Heart" as filled/active. **Testing Type:** Functional

Test Case ID: TC-FUNC-FAV-002

Objective: Verify an authenticated user can successfully remove a property from their favorites. **Preconditions:** The user is authenticated. A property is currently saved in the user's favorites list. **Test Procedure:** 1. Navigate to the frontend "Saved Properties" dashboard (or the property detail page). 2. Click the "Unsave" button. **Expected Result:** The frontend makes a **DELETE** request to `/api/property/:id/favorite`. The backend `unsaveProperty` controller removes the specific record from the `favorites` table. The server returns a **200 OK** status. The frontend successfully updates its state, removing the property card from the dashboard. **Testing Type:** Functional

Test Case ID: TC-FUNC-FAV-003

Objective: Verify unauthenticated users cannot interact with the favorites system.

Preconditions: The database is running. The browser session has NO authenticated user (no JWT in `localStorage`). A valid property exists. **Test Procedure:** 1. Using Playwright's API request context, send a **POST** request to `/api/property/:id/favorite` without the `Authorization` header. **Expected Result:** The backend authorization middleware intercepts the request. It returns a **401 Unauthorized** status before the controller is reached, protecting the endpoint from anonymous insertions. **Testing Type:** Security / Functional

Test Case ID: TC-FUNC-FAV-004

Objective: Verify the controller's error handling when provided with an invalid ID format for saving/unsaving. **Preconditions:** The user is authenticated. **Test Procedure:** 1. Send a **POST** or **DELETE** API request to an invalid route string, such as `/api/property/abc/favorite`.

Expected Result: The controller attempts to parse the ID, fails the `isNaN()` check, and safely returns a **400 Bad Request** with the message "Invalid property ID" without crashing the server. **Testing Type:** Functional

Test Case ID: TC-FUNC-MYLIST-001

Objective: Verify an authenticated user can successfully retrieve a list of properties they have published. **Preconditions:** The database is running. Two distinct users exist (User A and User B). User A has published two properties. User B has published one property. User A is authenticated. **Test Procedure:** 1. Navigate to the frontend "My Listings" dashboard. 2. Alternatively, fire an authenticated GET request to `/api/property/my-listings`. **Expected Result:** The backend `getMyProperties` controller successfully parses the user ID from the JWT. The server returns a `200 OK` status. The returned data array contains exactly the two properties owned by User A, completely excluding User B's property. The frontend successfully maps and renders these items. **Testing Type:** Functional

Test Case ID: TC-FUNC-MYLIST-002

Objective: Verify unauthenticated users cannot access the "My Listings" endpoint. **Preconditions:** The browser session has NO authenticated user (no JWT in `localStorage`). **Test Procedure:** 1. Using Playwright's API request context, send a GET request to `/api/property/my-listings` omitting the `Authorization: Bearer <token>` header. **Expected Result:** The backend authorization middleware intercepts the request. It returns a `401 Unauthorized` status before reaching the controller logic. **Testing Type:** Security / Functional

Test Case ID: TC-FUNC-MYSAVED-001

Objective: Verify an authenticated user can successfully retrieve their saved (favorited) properties. **Preconditions:** The database is running. User A is authenticated. Multiple properties exist in the database. User A has saved exactly one specific property to their favorites. **Test Procedure:** 1. Navigate to the frontend "Saved Properties" dashboard. 2. Alternatively, fire an authenticated GET request to `/api/property/saved`. **Expected Result:** The backend `getMySavedProperties` controller executes the `INNER JOIN` query between `properties` and `favorites` using User A's ID. The server returns a `200 OK` status with an array containing only the single property favorited by User A. **Testing Type:** Functional

Test Case ID: TC-FUNC-MYSAVED-002

Objective: Verify unauthenticated users cannot access the "Saved Properties" endpoint. **Preconditions:** The browser session has NO authenticated user. **Test Procedure:** 1. Using Playwright's API request context, send a GET request to `/api/property/saved` omitting the `Authorization` header. **Expected Result:** The request is blocked by the authorization middleware, returning a `401 Unauthorized` status. **Testing Type:** Security / Functional

Test Case ID: TC-FUNC-SEARCH-001

Objective: Verify search functionality successfully returns properties matching a partial string in the title or address. **Preconditions:** The database is running and contains at least two seeded properties: one with the title "Cozy Downtown Loft" and another at the address "123 Cozy Ave". **Test Procedure:** 1. Navigate to the frontend search page and type "cozy" into the search bar. 2. Alternatively, fire an unauthenticated `GET` request to `/api/property/search?q=cozy`. **Expected Result:** The backend `handleSearch` controller passes the query to the service. The `ILIKE` query successfully matches both the title of the first property and the address of the second. The server returns a `200 OK` status with an array containing exactly those two properties. The frontend renders the corresponding property cards. **Testing Type:** Functional

Test Case ID: TC-FUNC-SEARCH-002

Objective: Verify case-insensitive searching functions correctly. **Preconditions:** The database contains a property titled "LUXURY PENTHOUSE". **Test Procedure:** 1. Send a `GET` request to `/api/property/search?q=luxury penthouse`. **Expected Result:** The server returns a `200 OK` status with the "LUXURY PENTHOUSE" property included in the data array, proving that PostgreSQL's `ILIKE` operator successfully ignored the casing differences. **Testing Type:** Functional

Test Case ID: TC-FUNC-SEARCH-003

Objective: Verify system behavior when a valid search term yields zero results. **Preconditions:** The database is running but contains no properties related to "Castle". **Test Procedure:** 1. Send a `GET` request to `/api/property/search?q=Castle`. **Expected Result:** The database queries successfully but finds no matches. The server returns a `200 OK` status with an empty array `[]` for the data payload. The frontend gracefully displays an empty state message (e.g., "No properties found for 'Castle'"). **Testing Type:** Functional

Test Case ID: TC-FUNC-SEARCH-004

Objective: Verify controller error handling when the search query parameter is omitted. **Preconditions:** The backend server is running. **Test Procedure:** 1. Send a `GET` request to `/api/property/search` (without the `?q=` parameter). **Expected Result:** The `handleSearch` controller detects that `q` is undefined/falsy. It intercepts the request before hitting the database and returns a `400 Bad Request` status with the message "Search term is required". **Testing Type:** Functional